CS 3000 MIDTERM 2 — SIDE 1: Greedy Algorithms & Graph Traversals

Key Terms

Graph $G=(V,E)$: vertices V, edges E. *Directed* = one-way; *undirected* = bidirectional.
Weight $w(e)$: cost on an edge. Unweighted = all costs 1.
DAG: Directed Acyclic Graph — no directed cycles. Required for topo sort. Every tree is a DAG.
SCC: Strongly Connected Component — maximal set where every vertex reaches every other via directed paths. In a DAG, each node is its own SCC.
Condensation: contract each SCC to one node ⇒ always a DAG.
Topological order: linear order s.t. every edge $(u{\to}v)$ has u before v. Exists iff DAG. Unique iff Kahn's always has exactly 1 source.
Relaxation: if $\text{dist}[u]+w(u,v) < \text{dist}[v]$, update $\text{dist}[v]$. Core of Dijkstra and Bellman-Ford.
disc/fin: DFS clock starts at 1, +1 each discover or finish. $\text{fin}[v]$ set when DFS fully backtracks out of v. Max fin $= 2|V|$.
Greedy stays ahead: induction — show greedy's running measure ≥ OPT's at every step.
Exchange arg: take OPT, find first diff from greedy, swap to match greedy, show cost $\not\uparrow$, repeat.
Product state space: two agents ⇒ vertices = pairs (u,v), up to $|V|^2$ nodes. Dijkstra still valid if all weights ≥ 0.

GreedyJump — Algorithm

Heuristic: jump to farthest stone ≤10 ft away
Optimizes: (1) min jumps (2) max unused stones (3) max avg jump
Does NOT: min longest jump; min avg jump length

```
A=[0]; i=0; d(n+1)=inf                    % sentinel
while i < n:
j = i
while d(j)-d(i) <= 10:  j++               % overshoot
append j-1 to A                           % step back
i = j-1
return A
```

Inner loop exits when j is first stone *too far*, so $j{-}1$ is farthest reachable.
Example $d{=}[0,3,8,11,19,25,28]$:

```
i=0:  j stops at 3 (11>10) -> stone 2 (d=8)
i=2:  j stops at 4 (11>10) -> stone 3 (d=11)
i=3:  j stops at 5 (14>10) -> stone 4 (d=19)
i=4:  j stops at 7 (inf) -> stone 6 (d=28)
A = [0, 2, 3, 4, 6]
```

GreedyJump — Correctness Proof

Let $B[1\ldots k]$ = any OPT sequence, $A[1\ldots k']$ = greedy. Goal: $k' \leq k$.

Claim: For all $0 \leq \ell \leq k'$: $d(A[\ell]) \geq d(B[\ell])$

Base $\ell{=}0$: $d(A[0]) = 0 = d(B[0])$. ✓
IH: $d(A[\ell]) \geq d(B[\ell])$.
Step: $d(A[\ell{+}1]) = \max_i\{d(i) : d(i){-}d(A[\ell]) \leq 10\}$
Since $d(A[\ell]) \geq d(B[\ell])$, every stone reachable from $B[\ell]$ is reachable from $A[\ell]$, so:

$$d(A[\ell{+}1]) \geq \max_i\{d(i) : d(i){-}d(B[\ell]) \leq 10\} \geq d(B[\ell{+}1]) \checkmark$$

Conclude: greedy reaches shore in $\leq k$ steps ⇒ optimal.

Proof Technique Guide

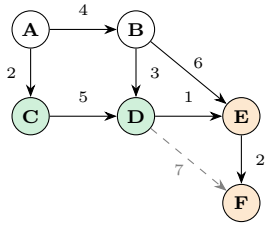
Greedy stays ahead:
1. Claim: $\forall\, 0 \leq \ell \leq k'$: [greedy] ≥ [OPT]
2. Base $\ell{=}0$: trivial
3. IH → step: greedy's local choice maintains lead
4. Conclude: $k' \leq k$

Exchange argument:
1. Find first index i where OPT ≠ greedy
2. Swap to match greedy ⇒ cost $\not\uparrow$
3. Repeat until OPT = greedy ⇒ greedy ≤ OPT

Use **stays-ahead** when greedy leads on a running measure. Use **exchange** when easier to compare element-by-element.

Master Graph + DFS (alpha from A)

Directed weighted DAG used for DFS, topo sort (Side 1) and Dijkstra (Side 2).



Clock rule: start at 1; +1 each discover or finish. Gray=open. Black=finished.

```
A(1)->B(2)->D(3)->E(4)->F(5)
F fin(6);E fin(7);D fin(8)
B->E: black->cross.  B fin(9)
A->C(10)->D: black->cross
C fin(11); A fin(12)
```

Node	disc	fin	Note
A	1	12	root, 1st tree
B	2	9	
D	3	8	
E	4	7	
F	5	6	leaf
C	10	11	root, 2nd tree

Tree edges: A→B, B→D, D→E, E→F, A→C. Cross: B→E, C→D. No back edges ⇒ DAG.

Topo order (decreasing fin): A(12)→C(11)→B(9)→D(8)→E(7)→F(6) ✓

Kahn's: remove in-degree-0 sources repeatedly.
Count orderings: multiply # sources at each step. Here always 1 ⇒ unique.

DFS Edge Types

- **Tree:** v undiscovered when (u,v) traversed
- **Back:** v is ancestor (gray) ⇒ **cycle!** ⇔ not DAG
- **Forward:** v descendant, already black
- **Cross:** v black, different subtree

Undirected DFS: only tree + back edges. When (u,v) seen and v visited, v must be ancestor since (v,u) already explored.

Graph T/F Traps

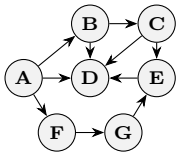
TRUE: $d(s,v) \leq R\ \forall v \Rightarrow d(u,v) \leq 2R$. $d(u,v) \leq d(u,s){+}d(s,v) \leq R{+}R{=}2R$ (triangle ineq.; undirected so $d(u,s) = d(s,u)$).

FALSE: $|d(u) - d(v)| = 1$ for all BFS edges. Counterex: triangle s,u,v — BFS gives $d(u) = d(v) = 1$, edge (u,v) gives 0. Correct: ≤ 1.

FALSE: SP stays SP in G' (all weights +1). Counterex: 2-hop 1+1=2 vs direct 2.5; in G': 4 vs 3.5 — flipped. k-hop path gains $+k$; more hops penalized more.

Exercise 2a Graph (A–G) — Official DFS

Edges: A→B, A→D, A→F, B→C, B→D, C→D, C→E, E→D, F→G, G→E. Alpha order from A.



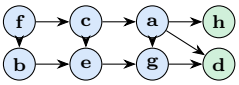
disc/fin: A(1/14) D(2/13) B(3/6) C(4/5) F(7/12) G(8/11) E(9/10)

```
A(1)->B(2?  No.  A's alpha nbrs:  B,D,F. Visits
B(2)
->B: alpha nbrs C,D. C(3)->C fin(4?  No:  C->D,E.
C(3)->D(4)->D fin(5)->C fin(6)->B fin(7)?
Re-trace official:  A(1)->D(2)->B(3)->C(4)
C fin(5)->B fin(6).  D->F(7)->G(8)->E(9)
E fin(10)->G fin(11)->F fin(12)->D fin(13)->A
fin(14)
```

Official answer: A visits D *second* (alpha: B then D), but then from D: B is unvisited, so D→B first. Trace: A(1)→D(2)→B(3)→C(4), C fin(5), B fin(6). Back to D→F(7)→G(8)→E(9), E fin(10), G fin(11), F fin(12), D fin(13), A fin(14).

Exercise 2c DAG — Official Topo Sort

f→c, f→b, c→a, c→e, a→h, a→g, a→d, b→e, e→g, g→d

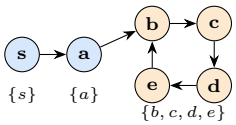


Official: f, c, a, h, b, e, g, d — exactly 1 valid ordering.

Only f has in-degree 0. After removing f, only c has in-degree 0 (b still needs f's removal only, so b *does* become available — but c must precede a,e and the chain forces a unique order with no parallel choices.

Exercise 2b — Official SCC Construction

$V{=}\{s,a,b,c,d,e\}$, $E{=}\{(s,a),(a,b),(b,c),(c,d),(d,e),(e,$



Cycle b→c→d→e→b forms SCC_3. 6V, 6E. DFS from s reaches all.✓
Kosaraju: (1) DFS on G, record fin order. (2) Reverse edges → G^R. (3) DFS on G^R in reverse fin order — each tree = one SCC. $O(V{+}E)$.

Connected Components

Undirected: BFS/DFS, count restarts = # components
Directed weak: ignore directions, treat undirected
Directed strong: Kosaraju ⇒ SCCs

- No back edges in DFS ⇔ DAG ⇔ topo sort exists
- Condensation (SCC → node) always a DAG
- Undirected BFS: non-tree edges connect same/adjacent levels only
- SCC decomposition of any directed graph is unique

CS 3000 MIDTERM 2 — SIDE 2: Shortest Paths, Dijkstra & Problem Modeling

Dijkstra on Master Graph (source A)

A→B(4), A→C(2), B→D(3), B→E(6), C→D(5), D→E(1), E→F(2), D→F(7)

```
Init:  A=0 B=inf C=inf D=inf E=inf F=inf
Pop A(0):  B->4, C->2
Pop C(2):  D->2+5=7
Pop B(4):  D->4+3=7(tie), E->10
Pop D(7):  E->7+1=8*, F->7+7=14
Pop E(8):  F->8+2=10*
Pop F(10):  done.
```

Node	dist	via	Note
A	0	—	source
B	4	A	
C	2	A	popped 2nd
D	7	B or C	two equal paths
E	8	D	updated from 10
F	10	E	D→F(14) beaten

Once popped, dist is final. Negative weights would let a popped node's dist improve later — breaking this. Hence $w \geq 0$ required.

Dijkstra — Algorithm & Runtimes

```
dist[s]=0; others inf
PQ = min-heap of all vertices
while PQ not empty:
u = extract-min(PQ)
for each neighbor v of u:
if dist[u]+w(u,v) < dist[v]:
dist[v] = dist[u]+w(u,v)
decrease-key(PQ,v,dist[v])
```

Why correct: when u popped, all PQ nodes have dist $\geq$ dist$[u]$; non-negative weights mean no future edge can improve dist$[u]$.

PQ	Runtime	Use when
Array	$O(V^2)$	dense $E{=}\Theta(V^2)$
Bin heap	$O((V{+}E)\log V)$	general
Fib heap	$O(E{+}V\log V)$	very sparse

Requires $w \geq 0$. Negative weights $\Rightarrow$ Bellman-Ford $O(VE)$.

Shortest Path Properties

- **Subpath**: every subpath of a SP is a SP
- **Triangle ineq.**: $d(s,v) \leq d(s,u) + w(u,v)$
- **Relaxation**: if dist$[u]$ correct, relaxing (u,v) gives correct SP through u
- **SP tree**: predecessor pointers form a spanning tree of SPs

Adding $+c$: k-hop path gains $+kc$. Different hop counts $\Rightarrow$ order can change $\Rightarrow$ SP not preserved.

Multiplying $\times c > 0$: all paths scale equally $\Rightarrow$ SP preserved.✓

BFS — Algorithm & Edge Property

```
BFS(G,s):  d[s]=0; Q=[s]
while Q not empty:
u = dequeue(Q)
for each neighbor v of u:
if v not visited:
d[v]=d[u]+1; enqueue(Q,v)
```

Output: shortest hop-count paths from s

Runtime: $O(V+E)$; Space: $O(V)$

Edge property: $|d(u) - d(v)| \leq 1$ for any edge (u,v) in undirected G.
Proof: if $d(v) \geq d(u)+2$, when u dequeued BFS would have set $d(v) = d(u)+1$ — contradiction. Same-level edges give 0; $= 1$ only for tree edges.

Exercise 4a — Mario (Single Agent)

Vertices: n islands

Edges: directed; door i from u to $a_{u,i}$ with weight $w_{u,i} \geq 0$; $E = 3n$

Algorithm: Dijkstra from s_1 to t

Runtime: $O(n \log n)$ time, $O(n)$ space $\quad [E = 3n = O(n)]$

Exercise 4b — Mario + Luigi Synchronized

Key insight: model joint state as a single vertex.

Vertices: pairs (u,v), u=Mario, v=Luigi. Up to n^2 nodes.

Edges: $(u,v) \to (a_{u,i}, a_{v,j})$ for all door pairs (i,j).

Weight: $\max(w_{u,i},\ w_{v,j})$ — slower agent is bottleneck.

$3{\times}3 = 9$ edges per node $\Rightarrow 9n^2$ edges total.

Source: (s_1, s_2) **Target:** (t,t)

Algorithm: Dijkstra $\Rightarrow O(n^2 \log n)$.

Why max? Both move simultaneously each step; pair waits for the slower one.

Why Dijkstra? $\max(w_{u,i}, w_{v,j}) \geq 0$ since all $w \geq 0$ — non-negativity preserved.

Exercise 5 — Bikeways

Vertices: t towns

Edges: directed bikeways with $\ell(a) \leq L$ only

Weights: $\ell(a)$ = length in miles

1. Keep only bikeways $\ell(a) \leq L$ $\quad O(p)$
2. Dijkstra from Centerville $\quad O(p \log t)$
3. Check all dist$[v] \leq M$ $\quad O(t)$

Total: $O(p \log t)$ (assumes $p \geq t$)

Why directed? Bikeways one-way only. **Why filter first?** Bikeways $> L$ can never be used; removing before Dijkstra shrinks the graph.

Problem Modeling Checklist

1. **States** → vertices (tuple for multi-agent)
2. **Transitions** → edges (directed if asymmetric)
3. **Costs** → weights (sum, max, etc.)
4. **Constraints** → filter edges or restrict states
5. **Goal** → source, target, algorithm

Goal	Algorithm	Req.
SP $w \geq 0$	Dijkstra	$w \geq 0$
SP neg.	Bellman-Ford	no neg. cycle
Unweighted SP	BFS	—
Reachability	DFS/BFS	—
Topo order	DFS fin times	DAG
SCCs	Kosaraju	directed
All-pairs SP	Floyd-Warshall	no neg. cycle

Quick Runtimes

Algorithm	Time	Space
BFS/DFS	$O(V{+}E)$	$O(V)$
Topo sort	$O(V{+}E)$	$O(V)$
Kosaraju SCC	$O(V{+}E)$	$O(V)$
Dijkstra (array)	$O(V^2)$	$O(V)$
Dijkstra (heap)	$O((V{+}E)\log V)$	$O(V)$
Dijkstra (Fib)	$O(E{+}V\log V)$	$O(V)$
Bellman-Ford	$O(VE)$	$O(V)$
Floyd-Warshall	$O(V^3)$	$O(V^2)$

Dense: array Dijkstra $O(V^2)$

Sparse $E{=}O(V)$: heap $\to O(V \log V)$

Mario single: $O(n \log n)$

Mario pair: $O(n^2 \log n)$

Bikeways: $O(p \log t)$

Greedy Proof Reference

Stays ahead (induction):

1. Define measure; claim: $\forall \ell \leq k'$: greedy $\geq$ OPT
2. Base ℓ=0: trivial (same start)
3. IH → step: local optimality maintains lead
4. Conclude: $k' \leq k$

Exchange argument:

1. First diff index i: swap to greedy, cost $\not\uparrow$
2. Repeat $\Rightarrow$ OPT = greedy $\Rightarrow$ optimal

GreedyJump uses stays-ahead (running distance measure), not exchange.

Common Mistakes

BFS: $|d(u) - d(v)| = 1$ is **FALSE**. Same-level edges give 0. Correct: ≤ 1.

Weight $+c$: does NOT preserve SPs. k-hop path gains $+kc$; more hops penalized more.

GreedyJump: inner loop overshoots (increments j first), then appends $j-1$. Stone 0 always in output.

Ex. 2a: official answer is D(disc=2). From A, alpha order is B,D,F — visits B first, then from B visits C. Back to A: then D. But official trace shows A→D(2). Verify directly with official solution.

DFS: max fin $= 2n$; disc $<$ fin always; no repeated fin times.

SCC Ex. 2b: 4-node SCC is cycle b→c→d→e→b, not pairs with back-edges.

GreedyJump: max avg jump IS optimized (3rd correct objective).

Exam Sanity Checks

- DFS: max fin $= 2n$; disc $<$ fin; no repeated times
- Topo: every edge goes left→right in ordering
- Dijkstra: popped nodes never re-enter PQ; dist only decreases
- SCC: DAG $\Rightarrow$ each node own SCC; condensation always DAG
- BFS: $|d(u) - d(v)| \leq 1$ all edges; $= 1$ only tree edges
- Runtime: state V, E in problem terms before formula
- Graph model: confirm directed vs. undirected, $w \geq 0$ for Dijkstra